

Triggers in Azure Data Factory (ADF)

In **Azure Data Factory (ADF)**, **triggers** are key components that define *when* and *how* a pipeline should run. They automate pipeline execution based on schedules, events, or dependencies, ensuring data workflows run consistently without manual intervention.

1. Schedule Trigger

The **Schedule Trigger** is the simplest and most widely used trigger type. It runs pipelines on a **fixed time-based schedule**, similar to a **cron job** or **task scheduler**.

- **Use Case:** Automating daily ETL loads, generating weekly reports, or executing hourly data refresh tasks.
- **Key Properties:**
 - **Start Date/Time & Time Zone:** Specifies when the trigger becomes active.
 - **Recurrence:** Defines how often the trigger runs (Minute, Hour, Day, Week, Month).
 - **End Date/Time (Optional):** Determines when the trigger stops executing.
- **Example:** Execute a pipeline every day at **2:00 AM UTC**.

2. Tumbling Window Trigger (TWT)

The **Tumbling Window Trigger** is a **stateful** trigger that processes data in **fixed-size, non-overlapping time intervals**. It ensures each time window is processed exactly once, supporting **idempotent** data handling.

Use Case:

Processing streaming telemetry data every 5 minutes, aggregating hourly sales, or performing historical data backfill.

Key Properties:

- **Start Time & Time Zone:** Marks the beginning of the first window.
- **Frequency & Interval:** Defines the size of each window (e.g., every 15 minutes, 1 hour, or 1 day).
- **Delay (Optional):** Waits for late-arriving data before execution.
- **Max Concurrency:** Controls how many windows can process simultaneously.
- **Stateful:** ADF keeps track of completed windows automatically.

Example:

A pipeline runs every 15 minutes to process website clickstream data for time windows like (00:00–00:15), (00:15–00:30), and so on. If a run fails, it can be re-executed for that *specific window*.

3. Event-Based Trigger

The **Event-Based Trigger** starts a pipeline when a specific **event** occurs, usually in **Azure Blob Storage** or **Azure Data Lake Storage (ADLS)**. It enables **real-time, data-driven automation**.

Use Case:

Triggering a pipeline automatically when a new file is uploaded or deleted from a storage container.

Key Properties:

- o **Linked Storage Account:** Defines which storage account is monitored.
- o **Event Types:**
 - Microsoft.Storage.BlobCreated
 - Microsoft.Storage.BlobDeleted
- o **Blob Path Begins With / Ends With:** Filters events by folder or filename patterns (e.g., /raw/input/ or .csv).
- o **Scope:** Can be configured at the container or account level.

Example:

When sales_20231027.csv arrives in the incoming folder, the pipeline automatically starts processing it.

4. Storage Event Trigger

This is a **specialized version** of the Event-Based Trigger, specifically focused on **Azure Blob Storage events**. Configuration, functionality, and use cases remain identical — it's simply a more targeted setup within the ADF interface.

5. Custom Event Trigger (Azure Event Grid)

The **Custom Event Trigger** listens for **user-defined or system events** published to **Azure Event Grid**, enabling deep integration with custom applications and other Azure services.

Use Case:

- o Triggering a pipeline when a **virtual machine** shuts down.
- o Running a data ingestion process when a **new order** is created by an external application.
- o Responding to custom events sent from **Logic Apps** or **Azure Functions**.

Key Properties:

- o **Subscription & Event Grid Topic:** Identifies where events are published.
- o **Event Type:** Filters events (e.g., MyApp.Orders.NewOrder).
- o **Subject Begins With / Ends With:** Filters by event subject patterns.

6. Manual Trigger (On-Demand Execution)

The **Manual Trigger** runs a pipeline **immediately** based on a **user action** or API call. It doesn't rely on a schedule or event.

Use Case:

- Running ad-hoc test executions, one-time data loads, or invoking pipelines programmatically.

Execution Methods:

- o Using **"Trigger Now"** from the ADF UI.
- o Via **PowerShell** (Invoke-AzDataFactoryV2Pipeline).
- o Through **REST API** or **SDKs** (.NET, Python, etc.).

7. Trigger Chaining (Dependency Trigger)

Though not listed as a separate trigger type, **Trigger Chaining** allows pipelines to execute **based on the completion or outcome** of another pipeline. It helps build **multi-step workflows**.

- **How It Works:** You can configure a trigger to depend on another pipeline’s **success, failure, or completion**.
- **Use Case:** Pipeline A cleans raw data, and Pipeline B starts automatically once Pipeline A completes successfully.

Summary Table

Trigger Type	Description	Typical Use Case	Stateful
Schedule	Executes pipelines on a fixed schedule.	Daily ETL, scheduled reports.	No
Tumbling Window	Runs for fixed, recurring time intervals.	Hourly/periodic batch processing.	Yes
Event-Based	Reacts to storage events like file arrival or deletion.	Real-time file-based processing.	No
Storage Event	Specialized Event-Based trigger for Blob Storage.	Automated blob ingestion.	No
Custom Event	Responds to user-defined or Azure service events.	App-driven or system-driven workflows.	No
Manual	Triggered immediately by user or API call.	One-time or ad-hoc runs.	No
Trigger Chaining	Executes based on another pipeline’s outcome.	Dependent pipeline orchestration.	No

Final Note

Choosing the right trigger depends on your **business logic** — whether execution should be **time-driven, event-driven, customized, or manual**.
A well-designed trigger strategy ensures pipelines run **reliably, efficiently, and in perfect synchronization** with your data workflow.